RABBIT Bipedal Robot

A MATLAB Simulation and Hybrid Zero Dynamics Gait Framework

Technical Documentation

Mohammad Malek

July 25, 2026

Abstract

This document describes a modular MATLAB framework for modeling, simulation, control, and trajectory optimization of the RABBIT planar five–link underactuated biped. The centrepiece is a working Hybrid Zero Dynamics (HZD) gait–optimization pipeline that produces dynamically feasible, periodic walking gaits and animates them. We cover the robot model, the continuous and hybrid dynamics, the HZD control formulation, the nonlinear program solved with `fmincon` in both of its transcriptions — single shooting and Hermite–Simpson direct collocation — the software architecture and data flow, a per–folder file reference, and reproducibility notes. Deviations between the documented and the as–built code (broken legacy entry points, orphaned files) are flagged explicitly.

1 Overview

RABBIT is a planar, five–link, *underactuated* biped: a torso plus two legs, each with a hip and a knee. Walking is modelled as a hybrid dynamical system — continuous single–support (swing) dynamics punctuated by a discrete impact when the swing foot strikes the ground. A gait is designed by imposing *virtual constraints* (B–spline reference trajectories for the actuated joints) and choosing their coefficients, together with the periodic initial state, so that the closed–loop system admits a stable periodic orbit while minimizing actuator effort.

Dependencies. MATLAB with the **Symbolic Math Toolbox** (only to *regenerate* the dynamics; the generated files are committed) and the **Optimization Toolbox** (`fmincon`, SQP — required for the gait pipeline).

Working entry point.

```
startup                                % add all folders to the path
cd Trajectory_Optimization
rabbit_hzd_trajectory_optimization     % optimize one gait, inspect, animate
```

A gait library across walking speeds is built with `build_gait_library.m`. See Section 13 for legacy entry points that no longer run.

2 Robot model

The generalized coordinates (7 DOF) are

$$q = \begin{bmatrix} p_x & p_z & q_t & q_1 & q_2 & q_3 & q_4 \end{bmatrix}^\top, \qquad x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix} \in \mathbb{R}^{14}. \tag{1}$$

symbol	meaning	actuated
p_x, p_z	torso base position (world frame)	no
q_t	torso pitch angle	no
q_1, q_2	stance–leg hip, knee	**yes**
q_3, q_4	swing–leg hip, knee	**yes**

Table 1: Generalized coordinates. Four actuators drive $q_1 \dots q_4$; the floating base (p_x, p_z, q_t) is unactuated, giving one degree of underactuation in single support.

The input map (`Dynamics/input_matrix.m`) is the constant

$$B = \begin{bmatrix} 0_{3\times 4} \\ I_4 \end{bmatrix} \in \mathbb{R}^{7\times 4}. \tag{2}$$

Physical parameters. There is no separate model/parameter file; the physical parameters are baked into the symbolic derivation (`Dynamics/rabbit_energy_model_generalized_Lagrange.m`, nested `Mass_Properties`): torso 10 kg / 0.75 m; each leg link (thigh, shank) 5 kg / 0.5 m; gravity $g = 9.8062\,\mathrm{m/s^2}$.

Sign convention (important). The world–frame Z axis is *up–positive* (ground = 0, above ground > 0), but p_z is stored *down–positive* in the generalized coordinates, so the world–frame hip height is $-p_z$. All kinematic helpers (`P_st`, `P_sw`, `Tt`, `get_body_points`) return world–frame, up–positive heights. Mixing the two conventions has historically caused sign bugs.

3 Continuous dynamics

The unconstrained equations of motion are

$$M(q)\,\ddot{q} + V(q,\dot{q}) + G(q) = B\,u, \tag{3}$$

with $M(q)$ the mass/inertia matrix (`Dynamics/M.m`), $V(q,\dot{q})$ the Coriolis/centrifugal vector (`Dynamics/V.m`, called as `V([q;dq])`), and $G(q)$ the gravity vector (`Dynamics/G.m`).

Single–support (constrained) dynamics. During swing the stance foot is pinned by the holonomic contact constraint $p_{\mathrm{st}}(q) = \text{const}$, i.e. at the acceleration level

$$J_{\mathrm{st}}(q)\,\ddot{q} + \dot{J}_{\mathrm{st}}\dot{q} = 0, \qquad J_{\mathrm{st}} = \frac{\partial p_{\mathrm{st}}}{\partial q} \in \mathbb{R}^{2\times 7}. \tag{4}$$

`Dynamics/rabbit_constrained_dynamics.m` solves the resulting saddle–point (KKT) system for the accelerations and the contact wrench λ:

$$\begin{bmatrix} M & -J_{\mathrm{st}}^{\top} \\ J_{\mathrm{st}} & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} Bu - V - G \\ -\dot{J}_{\mathrm{st}}\dot{q} \end{bmatrix}. \tag{5}$$

4 Hybrid model

Position is continuous across impact; only velocity jumps.

- **Guard / event** (`Contact/rabbit_impact_event.m`): the swing–foot height $p_{\mathrm{sw},z}(q)$ crosses zero from above.

- **Impact map** (`Contact/rabbit_impact_map.m`): momentum–conserving reset enforcing zero post–impact swing–foot velocity,

$$\begin{bmatrix} M & -J_{\text{sw}}^{\top} \\ J_{\text{sw}} & 0 \end{bmatrix} \begin{bmatrix} \dot{q}^{+} \\ \lambda \end{bmatrix} = \begin{bmatrix} M\,\dot{q}^{-} \\ 0 \end{bmatrix}. \tag{6}$$

- **Reset map** (`Reset_Map/rabbit_reset_map.m`): relabels stance/swing legs (`relabel_state.m`) and re–plants the new stance foot on the ground ($p_z \mathrel{+}= p_{\text{st},z}$). The horizontal coordinate is intentionally not shifted, so the robot walks forward continuously in the world frame.

5 Phase variable and Hybrid Zero Dynamics

Phase variable (linear in q). The gait "clock" is the absolute stance–leg angle, *linear* in q:

$$\theta(q) = q_t + q_1 + \tfrac{1}{2} q_2 = c\,q, \qquad c = \begin{bmatrix} 0 & 0 & 1 & 1 & \tfrac{1}{2} & 0 & 0 \end{bmatrix}. \tag{7}$$

This is implemented in `Controller/theta_of_q.m`; its gradient $\partial\theta/\partial q = c$ is the constant returned by `Controller/dtheta_dq_of.m`. Because both leg links have length $\frac{1}{2}$, the geometric hip–to–foot angle atan2 equals (7) exactly for all physical poses ($|q_2| < \pi$); the linear form is preferred because *it cannot wrap at* $\pm\pi$ and matches Westervelt's Hypothesis HH6, on which the HZD decoupling analysis relies.

Virtual constraints. The normalized phase is

$$s = \frac{\theta - \theta^{-}}{\theta^{+} - \theta^{-}} \in [0, 1], \tag{8}$$

where θ^{-} is the phase at step start and θ^{+} (`p.theta_plus`) at touchdown. The four actuated joints track a clamped cubic B–spline reference $y_d(s)$,

$$y(q) = q_{4:7} - y_d(s), \qquad u = -K_p\,y - K_d\,\dot{y}, \tag{9}$$

with $K_p = 400$, $K_d = 40$. The B–spline is evaluated by `bspline_eval.m` (basis functions from `BSpline.m` / `BSpline_derivative.m`); the whole closed loop is assembled in `Dynamics/hzd_closed_loop_ode.m` whose augmented state $\xi = [x;\ \text{torque cost}]$ integrates the running cost alongside the dynamics.

6 Trajectory optimization (the nonlinear program)

Decision vector.

$$z = \begin{bmatrix} \text{vec(coeffs)} \\ x_{\text{start}} \end{bmatrix} \in \mathbb{R}^{\,n_u n_c + 14} = \mathbb{R}^{38}, \tag{10}$$

with $n_u = 4$ actuated outputs and $n_c = 6$ control points each (`unpack_z.m`).

Cost (`hzd_cost.m`): squared torque per unit step length (Westervelt eq. 6.43),

$$J(z) = \frac{1}{L_{\text{step}}} \int_0^{T_{\text{step}}} \|u(t)\|_2^2\,dt, \tag{11}$$

where L_{step} is the stance–to–swing foot distance at impact. Normalizing by L_{step} removes the degenerate incentive to "step in place".

Constraints (`hzd_constraints.m`). Equalities (ceq $\in \mathbb{R}^{18}$):

- one–step periodicity, $x_{\text{next}}(2{:}14) = x_{\text{start}}(2{:}14)$, with $x_{\text{next}} = \Delta_{\text{reset}}\big(\Delta_{\text{impact}}(x_{\text{end}})\big)$ (13);
- stance foot on ground at $t = 0$ (1);
- stance foot zero velocity, $J_{\text{st}}\dot{q}_{\text{start}} = 0$ (2);
- touchdown phase, $\theta(q_{\text{end}}) = \theta^{+}$ (1);
- **NEC1** average walking rate, $L_{\text{step}}/T_{\text{step}} = v_{\text{des}}$ (1).

Inequalities ($c \in \mathbb{R}^{8}$, $c \le 0$): no ground penetration; mid–step swing–foot clearance (NIC3); step–length floor; orbital stability (Section 8); and the four *physical realizability* constraints of Section 9 (torque, friction, ground reaction, impact impulse). The stability and physical constraints are each individually *gated* (default off) — see those sections. Disabled inequalities are held at a trivially satisfied value so c keeps constant length.

Solver (`hzd_problem_setup.m`). `fmincon` SQP with *central* finite differences (the event–triggered `ode45` makes forward differences noisy) and `ScaleProblem` *off* (so reported feasibility is directly comparable to the constraint tolerance). Cold starts from x_0 are unreliable; `build_gait_library.m` warm–starts each speed from the previously converged gait and marches v_{des} in small steps.

Diagnostics. `inspect_solution.m` prints and returns a full breakdown of any z: the θ sweep, step length/duration, walking speed, cost, every constraint residual, the worst violation, the within–step stance–foot drift, and the orbital–stability radius ρ (below).

7 Direct collocation

Everything above transcribes the gait problem by *single shooting*: the decision variables are the Bézier coefficients and one initial state, and the dynamics are satisfied implicitly because `ode45` integrates them. `Trajectory_Optimization/Collocation/` implements the alternative transcription — *Hermite–Simpson direct collocation* — which matches the thesis method. There is no `ode45` in the optimization loop at all.

What changes. The trajectory itself becomes a decision variable. For N nodes the vector z concatenates

block	size	meaning
X	$2n_q \times N$	node states $[q; \dot{q}]$
U	$n_u \times N$	node joint torques
Λ	$2 \times N$	node stance–foot contact forces $[F_x; F_z]$
`coeffs`	$n_u \times n_c$	Bézier virtual–constraint parameters
T	1	step duration

(`col_pack.m` / `col_unpack.m`). With the defaults $n_q = 7$, $n_u = 4$, $n_c = 6$, $N = 20$ that is $280 + 80 + 40 + 24 + 1 = 425$ variables, versus the 38 of the shooting formulation. The dynamics are then imposed as *equality constraints* — the Hermite–Simpson defects between consecutive nodes — rather than being integrated. The virtual constraints $y = q_{4:7} - y_d(s, \texttt{coeffs}) = 0$ are enforced at *every* node, so the solution lives on the hybrid zero–dynamics manifold by construction and `coeffs` still parametrizes it.

The trade. Collocation buys a much better conditioned problem: no event–triggered integrator inside the finite differences, so the Jacobian is sparse, exact in structure, and free of the integration noise that forced central differences in the shooting setup. It pays for this with an order of magnitude more variables and with a solution that is only *as feasible as its defect tolerance* — which motivates the cross–check below.

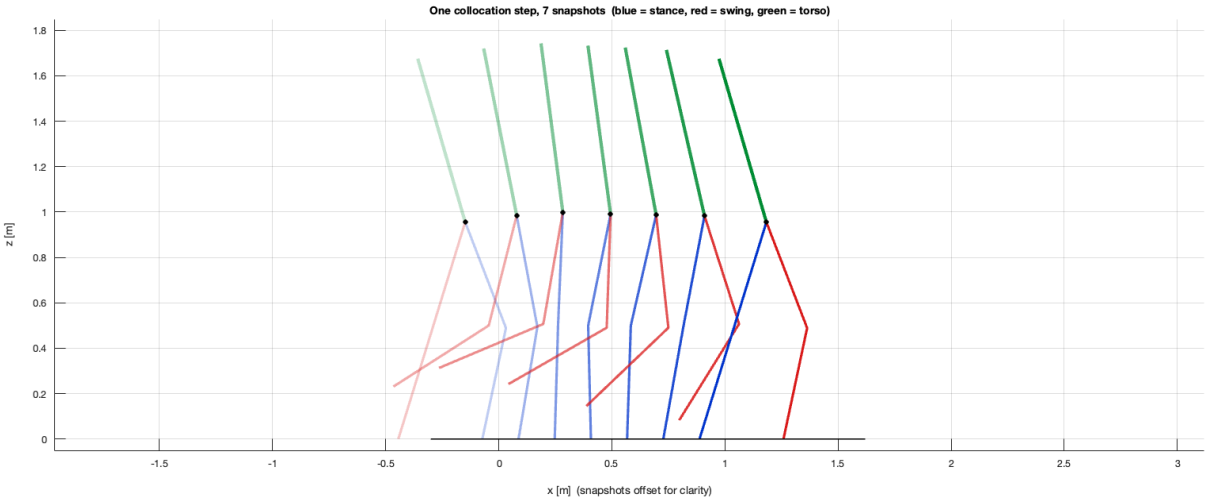

Figure 1: One converged collocation step (`col_result_2026-07-23_00-45-04.mat`, $N = 20$, $T = 1.046$ s), sampled at seven nodes and offset horizontally for legibility. Blue is the stance leg, red the swing leg, green the torso; opacity increases through the step. The stance foot stays planted while the swing leg advances and re–strikes ahead of it.

Warm starting (`rabbit_hzd_collocation.m`). The driver accepts either kind of saved gait and does the right thing with each:

- a *collocation* result (`col_result_*.mat`, identified by carrying an `N` field) has its z reused directly — N must match;
- a *shooting/PD* result (`hzd_result_*.mat`) is simulated for one step and resampled onto the N nodes by `col_seed_from_pd.m`.

This is what makes the "one constraint at a time" workflow practical: enable one physical constraint, re–solve warm–started from the last converged collocation gait, repeat. `col_verify_seed.m` reports the norm of every equality block on a seed *without solving*, which separates a genuine transcription bug from a merely hard NLP.

Cross–checking the result (`col_crosscheck.m`). A collocation solution is an **open–loop** trajectory. It satisfies the defect constraints, but nothing in the transcription guarantees that the shooting simulator, driven by an actual feedback law, reproduces it. The cross–check extracts `coeffs` and the node–1 state, simulates one step under `p.controller` (`'pd'` or `'clfqp'`), and overlays the two (Figure 2).

Stability of a collocation gait (`col_poincare.m`). Same Poincaré machinery as Section 8, fed from a `col_result_*.mat`. Use `'pd'`: the CLF–QP law solves a QP per ODE step, and ρ costs ≈ 26 step simulations.

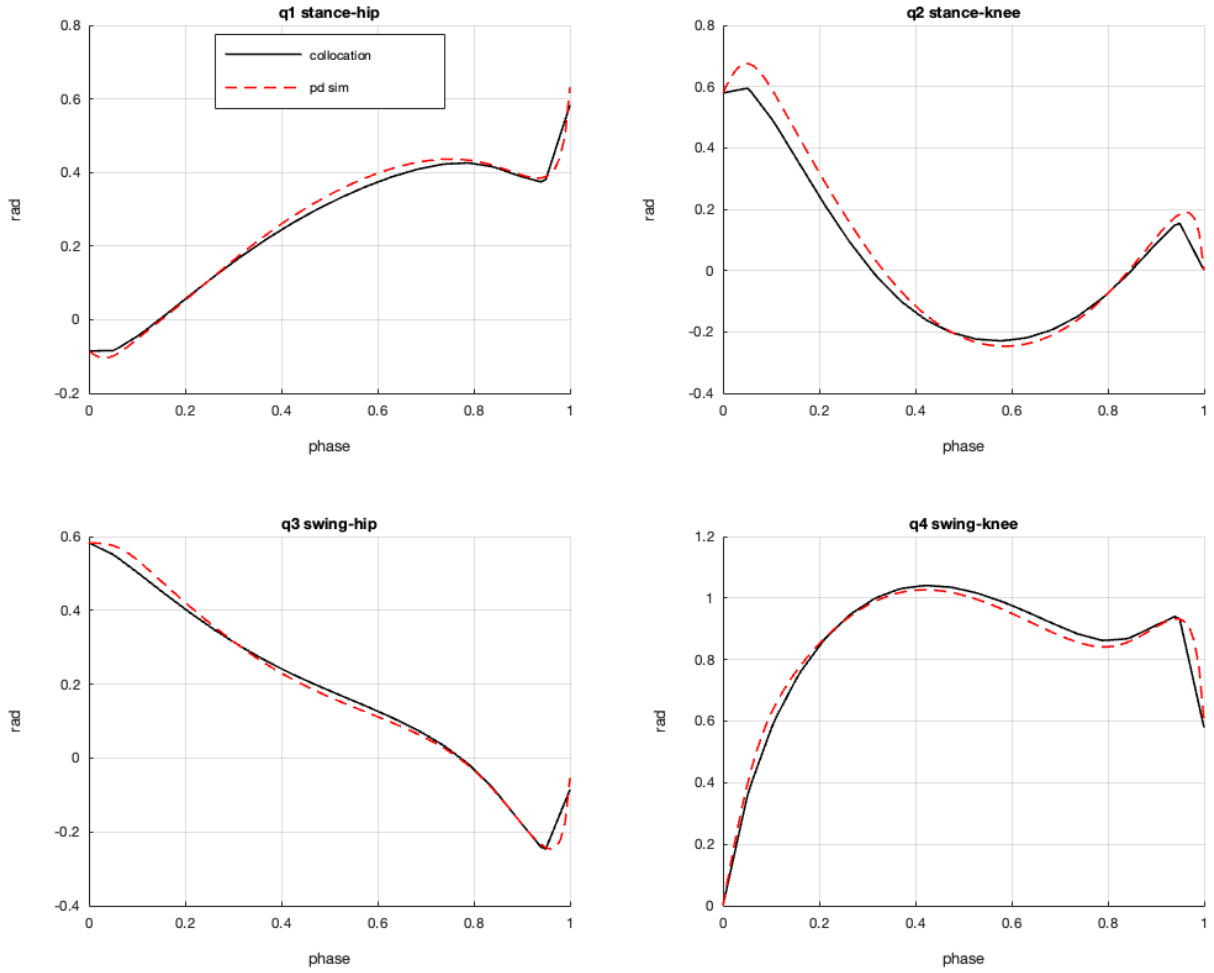

Figure 2: Collocation reference (black) versus the PD closed–loop simulation (red dashed) over one step, on a normalized phase axis. Tracking is close — 0.026 rad rms, 0.140 rad max, the largest errors at the stance knee just after impact — so the open–loop trajectory really is realizable by the feedback law. Note this says nothing about what happens on the *next* step; see Figure 3.

8 Orbital stability: periodic is not the same as stable

This section is deliberately expanded for teaching, because it is the single most instructive point in the pipeline.

The trap. The periodicity equality makes x_{start} a *fixed point* of the step–to–step (Poincaré) map

$$x_{\text{next}} = P(x_{\text{start}}) := \Delta_{\text{reset}}\big(\Delta_{\text{impact}}(\varphi_{\text{step}}(x_{\text{start}}))\big), \tag{12}$$

where φ_{step} is one closed–loop swing phase. But *a fixed point can be unstable.* Satisfying periodicity to machine precision guarantees a limit cycle *exists*; it says nothing about whether the robot stays on it.

The diagnosis. Linearize (12) about the fixed point, $A = \partial P/\partial x$, and look at its spectral radius

$$\rho = \max_i |\lambda_i(A)|. \tag{13}$$

The gait is orbitally (asymptotically) stable iff $\rho < 1$. This is Westervelt's stability requirement (informally, NEC5). For the reference seed gait obtained by enforcing periodicity *alone*, the numbers are stark:

$\max \lvert\text{periodicity}\rvert$	ρ	open–loop $\lvert\dot{q}\rvert$ over 6 steps
0.007 (excellent)	≈ 3 (unstable)	$6 \to 6 \to 19 \to 891 \to 949 \to 440$

The orbit is a near–perfect fixed point that the robot falls off after two steps. Every downstream symptom we observed — stance–foot drift growing across a step, Baumgarte constraint stabilization diverging, the Poincaré settling iteration blowing up, fragile speed continuation — is the same instability seen from a different angle: an orbit with $\rho > 1$ amplifies *every* perturbation, numerical or physical.

The remedy. Add $\rho < 1$ as an optimization constraint. `poincare_stability.m` builds A by central finite differences on the 13 periodic states (index $2:14$; p_x is the translation gauge and is excluded),

$$A_{:,j} = \frac{P(x_{\text{start}} + h\,e_j) - P(x_{\text{start}} - h\,e_j)}{2h}, \qquad j = 1, \ldots, 13, \tag{14}$$

and returns $\rho = \max_i |\lambda_i(A)|$. `hzd_constraints.m` then adds the inequality $\rho - \rho_{\max} \le 0$ (with $\rho_{\max} = 0.95$).

Cost and workflow. Each ρ evaluation costs $\approx 2 \times 13 = 26$ step simulations, and `fmincon` finite–differences the constraint over 38 variables — hundreds to thousands of simulations per iteration. The constraint is therefore **gated** behind `p.enforce_stability` (default `false`). The recommended workflow is two–phase:

1. solve a *periodic* gait with stability off (fast);
2. turn `p.enforce_stability = true` and re–solve *warm–started* from that gait to drive ρ below 1.

`inspect_solution.m` reports ρ on every call regardless of the gate, so stability is always visible even when it is not being enforced.

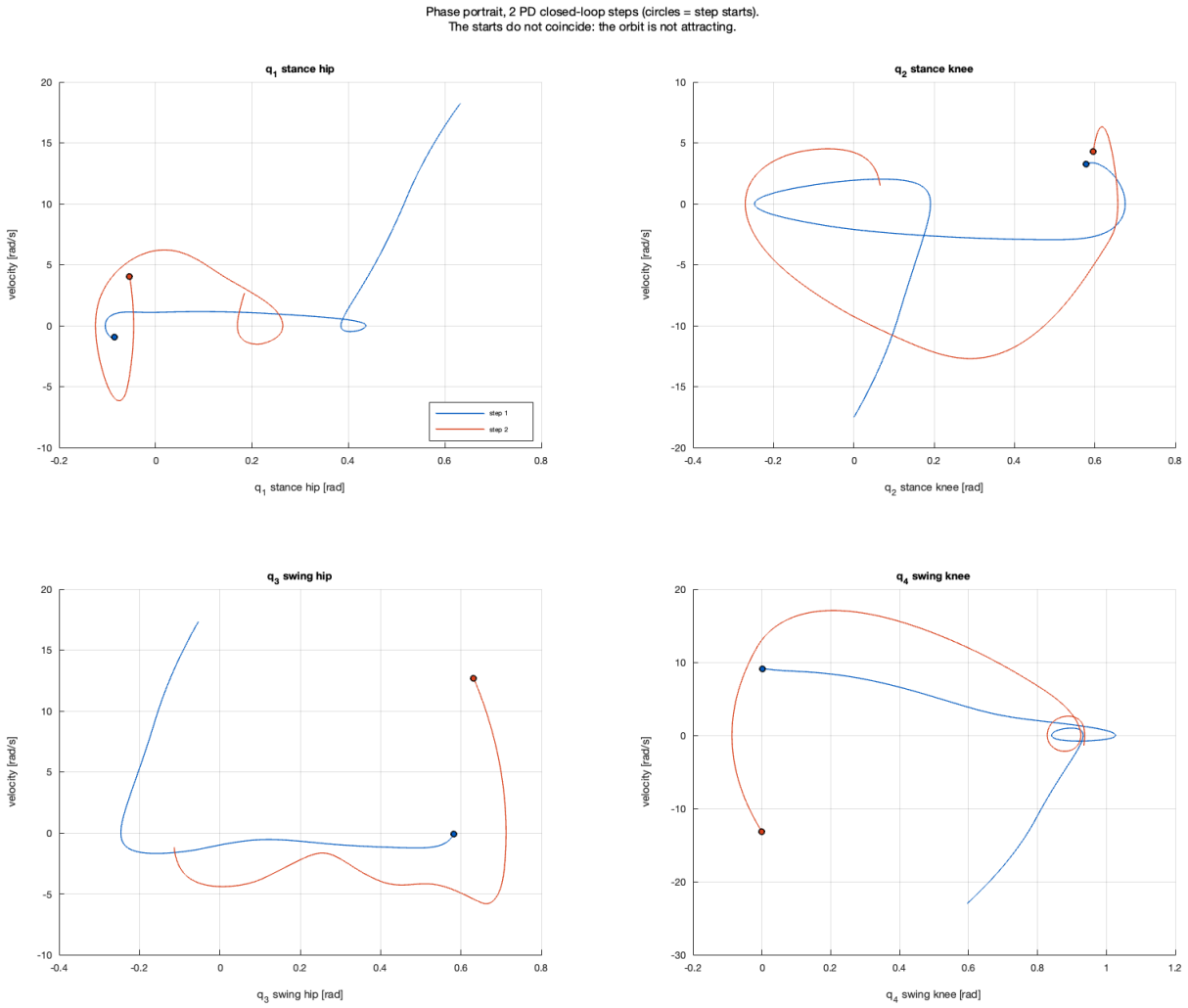

Figure 3: What $\rho > 1$ looks like. Phase portraits of the four actuated joints over consecutive PD closed–loop steps, started from the collocation gait's node–1 state. Circles mark where each step *begins*: on an orbitally stable limit cycle they would coincide, and instead step 2 starts visibly away from step 1 — most starkly at the swing hip, which restarts with ≈ 12.7 rad/s where step 1 began near zero. Only two steps are drawn because the third leaves the physical range entirely ($|q|$ reaching several hundred radians), the same blow–up tabulated above. Compare Figure 2: a *single* step tracks its reference well, which is exactly why one–step agreement is not evidence of a walkable gait.

Analogy. Just as the *step–length floor* was the missing constraint that stopped the optimizer exploiting a degenerate "step in place", the *stability bound* is the missing constraint that stops it returning a mathematically periodic but physically un–walkable orbit. In both cases the optimizer was doing exactly what it was told; the specification was incomplete.

9 Physical realizability (thesis Table 3.1)

Two families of constraints. It helps to see the whole set as doing two different jobs:

- **Mathematical validity** — is this a *real, attracting* gait? Periodicity (a fixed point), $\theta_{\text{end}} = \theta^+$ (the phase closes), NEC1 (walking rate), and orbital stability ($\rho < 1$, Section 8). These say nothing about physics: a gait can satisfy all of them and be impossible to build.
- **Physical realizability** — can the hardware and the ground actually produce it? The Table 3.1 constraints below.

Both are required, and "periodic" (cheap to satisfy) is a tiny subset of "walkable" (both families). All of the physical quantities are recovered from the *same* simulation the optimizer already runs: the contact force $\lambda = [F_x; F_z]$ is an output of `rabbit_constrained_dynamics`, the joint torque u of `hzd_control_and_dynamics`, and the impact impulse is the multiplier inside `rabbit_impact_map`. `gait_forces.m` replays the trajectory and returns the peaks; `hzd_constraints.m` turns them into inequalities.

constraint	form (value)	what failure it prevents
Vertical GRF	$\min F_z \geq F_{z,\min}$ (≥ 20 N)	ground can only *push* (unilateral contact); $F_z \leq 0$ means the foot lifts / the model invents a pulling force
Friction cone	$\max \lvert F_x/F_z \rvert \leq \mu$ (0.4)	stance foot *slipping* (well posed only where $F_z > 0$)
Joint torque	$\max \lvert u \rvert \leq u_{\max}$ (120 Nm)	commanding torque the actuator cannot deliver
Impact impulse	$\lvert F_e \rvert \leq F_{e,\max}$ (30 Ns)	a violent foot–strike (hardware damage, hard landing)
Swing clearance	$h_f \geq h_{\min}$ (0.1 m, NIC3)	scuffing / premature re–contact (a "step in place"); ensures S is hit once, at touchdown
Dynamic constr.	EoM	(free here — we integrate the ODE, so it holds by construction)

Values do not transfer from the reference. The thesis numbers ($|u| \leq 5$ Nm, $F_z \geq 200$ N, $F_e \leq 15$ Ns) are for ATRIAS: **63 kg with 50:1 harmonic drives**, so its 5 Nm *motor* torque is 250 Nm at the joint. RABBIT here is ∼30 kg with **no gear ratio** (u is the joint torque directly), so the limits must be re–derived — measure first (`inspect_solution` prints all four), then set bounds.

Order matters — a worked diagnosis. Measured on the reference gait, the quantities were:

quantity	measured	verdict
peak $\lvert u\rvert$	257 Nm	high ($> u_{\max}$)
mean F_z	+309 N	$\approx$ body weight, correct sign
min F_z	$\mathbf{-193}$ N	**negative: the foot lifts off**
max $\lvert F_x/F_z\rvert$	255	artifact of $F_z \to 0$ (not real slip)
$\lvert F_e\rvert$	9.8 Ns	fine

The instructive point: the enormous friction ratio is *not* a friction problem. $|F_x/F_z|$ diverges wherever F_z crosses zero, and here F_z goes negative — physically impossible, since the ground can only push. So the *ground–reaction* constraint ($F_z > 0$, foot in compression) is the root; the friction ratio only becomes meaningful once it holds (and `gait_forces` therefore samples $|F_x/F_z|$ only where $F_z > 1$ N). Enabling friction first (before fixing F_z) made `fmincon` chase a divide–by–zero and diverge; enabling *GRF first* is correct. Each constraint has its own toggle (`p.enforce_grf`, `enforce_friction`, `enforce_torque`, `enforce_impulse`) precisely so they can be introduced one at a time, warm–started, in the physically sensible order.

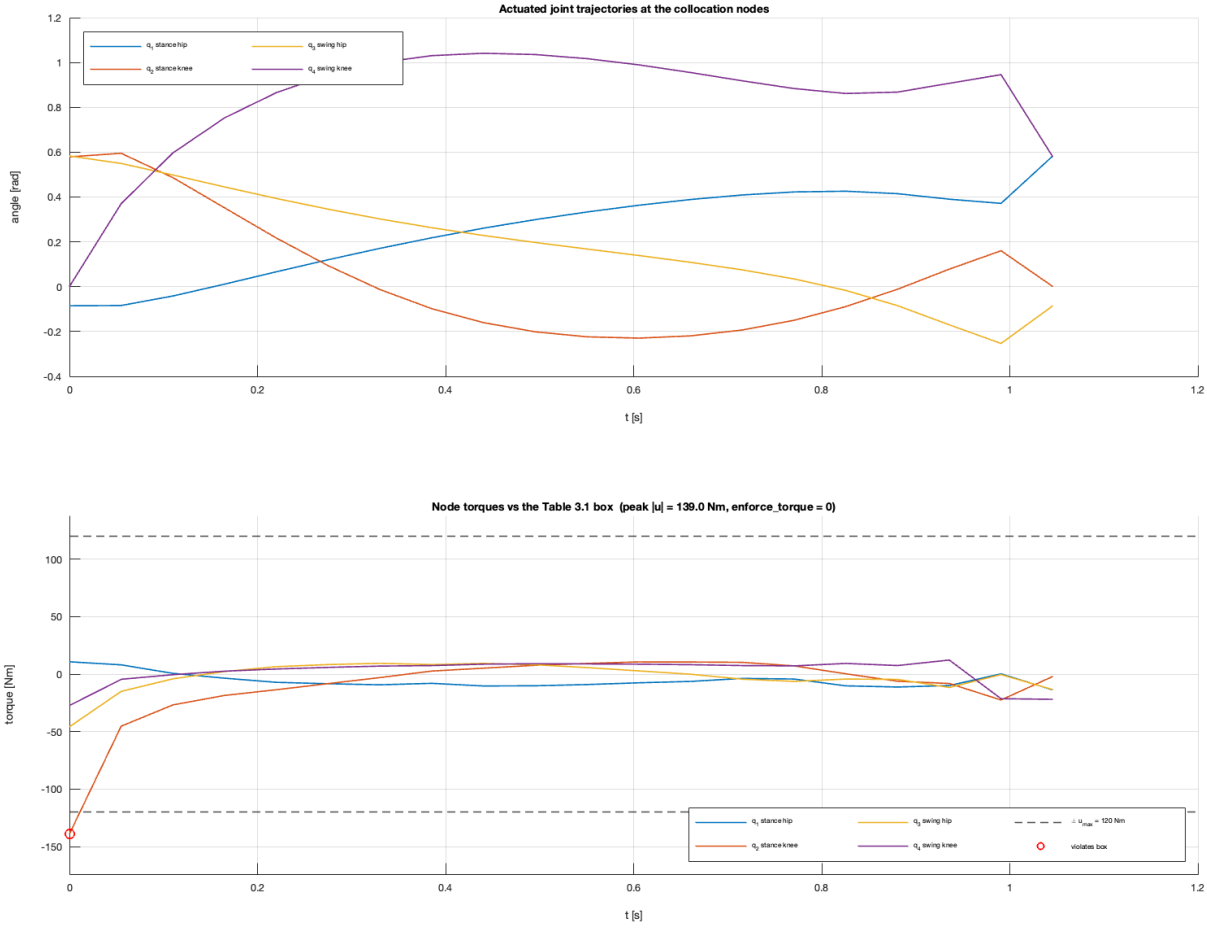

Figure 4: A gait caught *mid–way* through that one–at–a–time workflow. The saved collocation result has `enforce_grf` and `enforce_impulse` on but `enforce_torque` still **off**, and the lower panel shows the consequence: exactly one of the 80 node torques leaves the ± 120 Nm box (circled) — the stance knee at the first node, at -139.0 Nm, a 16% overshoot. With the toggle off this is not a solver failure but an *unconstrained* quantity, and it is precisely what enabling `enforce_torque` on the next warm–started solve is there to remove. The remaining 79 nodes sit well inside the box, which is why the violation is easy to miss without plotting it.

The constraints are coupled, not four independent knobs. Two couplings are worth internalizing. First, *GRF is upstream of friction*: the ratio $|F_x/F_z|$ diverges as $F_z \to 0$, so friction is only well posed once $F_z > 0$ is enforced — the ordering is dictated by the math, not taste. Second, *torque is upstream of almost everything*: a tighter torque budget caps joint accelerations, which caps how hard the robot throws its mass, which shrinks GRF spikes and friction demand. A violent gait therefore violates the whole table at once — which is exactly what the reference seed does (257 Nm, F_z changing sign, $\rho \approx 129$). These are four views of one question: *is this gait gentle enough to be real?*

Caveat. The reference gait violates almost every one of these by large margins *and* is strongly unstable ($\rho \gg 1$). Repairing it incrementally may not converge from such a pathological seed; a materially better initial gait, or the reduced 2–D zero–dynamics formulation, is the likely path to a gait that is simultaneously periodic, stable, and physically realizable.

Synthesis: constraints as specification completeness. Every constraint beyond the raw equations of motion was added because the optimizer found a loophole in an incomplete specification, and each loophole was a *different* flavour of "technically periodic":

1. periodic but **degenerate** (step in place) → step–length floor;
2. periodic but **unstable** (falls after two steps) → stability, $\rho < 1$;
3. periodic but **unphysical** (foot lifts, slips, over–torque) → Table 3.1.

The optimizer was never wrong; it did exactly what it was told, and each surprising result exposed a missing line in the specification. Table 3.1 is best read not as "extra realism" bolted on, but as *the rest of the definition of walking* — the part that is obvious to a human and invisible to `fmincon` until it is written down.

10 Software architecture and data flow

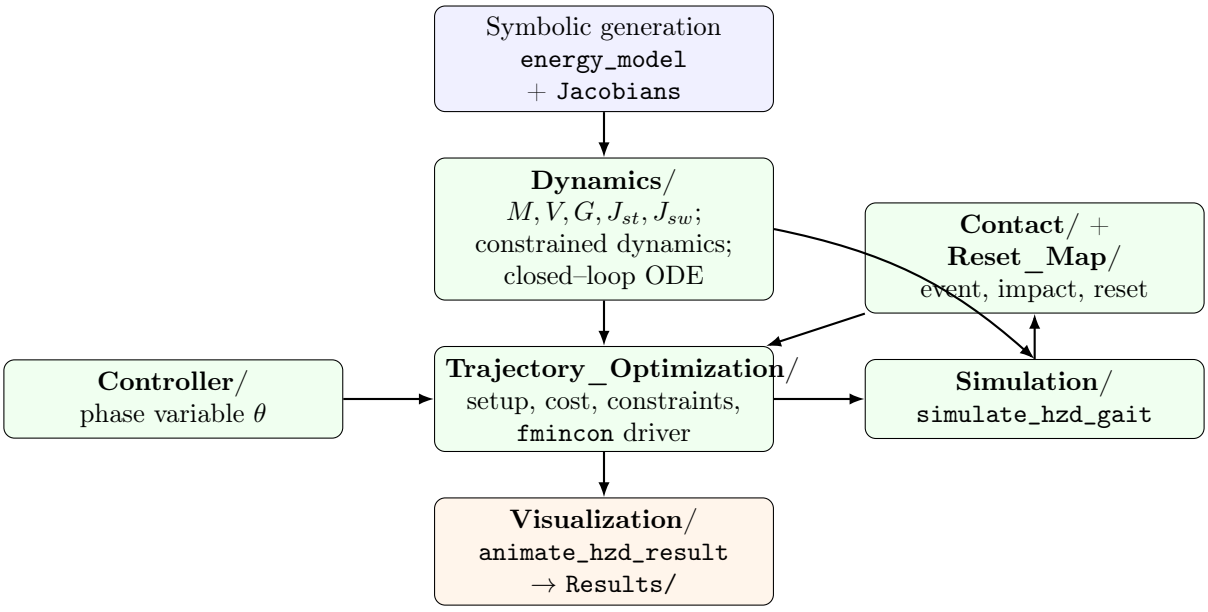

Figure 5: Pipeline: offline symbolic generation feeds the **Dynamics** layer; the optimizer repeatedly calls the single–step simulator (which integrates the closed–loop dynamics and detects impact) to evaluate the cost and constraints; the converged gait is animated and saved.

11 File reference

Dynamics/ (mostly Symbolic–Toolbox generated)

file	role
`rabbit_energy_model_generalized_coordinates.m`	Generalized Lagrange–Euler derivation; defines masses/inertias; produces `Tt,T1-T4,P_st,P_sw,M,DM,V,G`.
`Jacobians.m`	Generator: produces `J_st,J_sw,Jdotdq_st,Jdotdq_sw`.
`Tt.m`, `T1-T4.m`	Homogeneous transforms of torso / link frames, `T=Tt(q)`.
`P_st.m`, `P_sw.m`	Stance / swing foot position $[x; z]$, world frame.
`M.m`	Mass matrix $M(q)$ (7×7).
`V.m`	Coriolis/centrifugal vector, `V([q;dq])`.
`G.m`	Gravity vector $G(q)$.
`DM.m`	$\dot{M}$, `DM([q;dq])` (used to form V).
`J_st.m`, `J_sw.m`	Foot Jacobians (2×7).
`Jdotdq_st.m`, `Jdotdq_sw.m`	$\dot{J}\dot{q}$ terms (2×1).
`input_matrix.m`	Constant actuation map B.
`rabbit_constrained_dynamics.m`	Single–support forward dynamics (KKT solve), `[ddq,lambda]=...(q,dq,u)`; optional Baumgarte via a 4th `foot_ref` arg.
`hzd_control_and_dynamics.m`	Control law + constrained dynamics for one state; returns `[ddq,tau,lambda]` (lets torque/force be replayed post–sim).
`hzd_closed_loop_ode.m`	Closed–loop swing RHS; wraps the above and appends the torque–cost integrand.
`rabbit_ode.m`	Generic single–support RHS for an arbitrary controller.
`rabbit_dynamics.m`	**Broken**: calls nonexistent `D_matrix/C_vector/G_vector`.

Contact/ and Reset_Map/

file	role
`rabbit_impact_event.m`	ODE event: swing foot reaches ground.
`impact_event_wrapper.m`	Same event for the augmented HZD state ξ.
`rabbit_impact_map.m`	Momentum–conserving velocity reset, `[x_plus,impulse]=...(x_minus)` (impulse is the optional 2nd output).
`relabel_state.m`	Swap stance/swing joints and velocities.
`rabbit_reset_map.m`	Relabel + re–plant stance foot on ground.

Controller/

file	role
`theta_of_q.m`	Phase variable $\theta = q_t + q_1 + \frac{1}{2}q_2$.
`dtheta_dq_of.m`	Constant gradient c.
`rabbit_controller.m`	**Broken**: calls nonexistent `desired_gait`.

`Trajectory_Optimization/` (main pipeline)

file	role
`rabbit_hzd_trajectory_optimization.m`	**Entry point**: single–gait solve, save, animate.
`build_gait_library.m`	Speed continuation; builds a `gait_library` struct.
`hzd_problem_setup.m`	Single source of p, bounds, `fmincon` options.
`hzd_cost.m`	Objective (torque^2 per step length).
`hzd_constraints.m`	Equality + inequality constraints.
`unpack_z.m`	Split $z = [\text{coeffs}; x_{\text{start}}]$.
`make_coeffs.m`	Initial B–spline control points.
`bspline_eval.m`	Evaluate a spline output and its s–derivative.
`BSpline.m`, `BSpline_derivative.m`	Cox–de Boor basis and derivative.
`inspect_solution.m`	Full diagnostic breakdown of a solution (incl. ρ).
`poincare_stability.m`	Step–map Jacobian; returns spectral radius ρ (NEC5).
`gait_forces.m`	Replays a gait; returns peak torque, min/mean vertical GRF, friction ratio, impact impulse (Table 3.1).
`settle_initial_state.m`	Poincaré iteration (disabled by default).

`Trajectory_Optimization/Collocation/` (Section 7)

file	role
`rabbit_hzd_collocation.m`	**Entry point**: Hermite–Simpson collocation solve, warm–started from a PD or collocation gait.
`col_problem_setup.m`	Extends p with the node count N and the collocation–vector bounds.
`col_pack.m`, `col_unpack.m`	Assemble / split $z = [X; U; \Lambda; \texttt{coeffs}; T]$.
`col_dynamics.m`	Constrained EoM evaluated at a node (state, torque, contact force).
`col_cost.m`	Objective on the node torques.
`col_constraints.m`	Hermite–Simpson defects, virtual constraints at every node, foot pinning, periodicity.
`col_seed_from_pd.m`	Simulate a PD gait and resample it onto N nodes to build z_0.
`col_verify_seed.m`	Report every equality–block norm on a seed *without* solving (transcription sanity check).
`col_crosscheck.m`	Open–loop collocation trajectory vs. a closed–loop sim (Figure 2).

file	role
`col_poincare.m`	Spectral radius ρ of a collocation gait under `'pd'` or `'clfqp'`.

Simulation/, Utilities/, Visualization/

file	role
`simulate_hzd_gait.m`	One HZD step, summary outputs (status, penetration, clearance, T_{step}).
`simulate_hzd_gait_full.m`	One HZD step, full trajectory (for animation/drift).
`simulate_one_step.m`	Generic one step for an arbitrary controller.
`simulate_n_steps.m`	Chain generic steps with impact+reset.
`get_body_points.m`	World positions of key body points.
`check_ground_validity.m`	Ground/penetration/slip validation of a state.
`animate_rabbit.m`	Stick–figure animation + GIF export.
`animate_hzd_result.m`	Stitch n HZD steps and animate.

config/ and Test/

file	role
`make_initial_state.m`	Deterministic ground–valid start state (1 output).
`make_random_initial_state.m`	Rejection–sampled random start (unseeded).
`config.m`	Singleton: stepping–stone terrain + spline params.
`init_bspline_params.m`	Legacy B–spline controller config.
`test_simulate_steps.m`	Smoke test of the generic simulation path.

12 Reproducibility

- **Generated dynamics.** `Tt,T1-T4,P_st,P_sw,M,DM,V,G` come from `rabbit_energy_model_generalized` `J_st,J_sw,Jdotdq_st,Jdotdq_sw` from `Jacobians.m`. *If any mass, length, inertia, or kinematic relation changes, these generator scripts must be re–run* to regenerate the committed files. Do not hand–edit the generated files.

- **Saved results.** `Results/hzd_result_<timestamp>.mat` each store `z_opt, p, fval, exitflag, report`. The `2026-07-20_12-06-36` result is the reference periodic gait ($\approx$ 0.355 m/s) used as the warm–start seed. `build_gait_library.m` writes `gait_library_<timestamp>.mat`. Collocation solves (Section 7) write `Results/col_result_<timestamp>.mat`, which additionally carry `N` — that field is how `rabbit_hzd_collocation.m` tells a collocation warm start from a PD one.

- **Default result selection is by filename, not mtime.** Every entry point that resolves a default `Results/*.mat` — `rabbit_hzd_collocation`, `col_crosscheck`, `col_poincare`, `col_verify_seed`, `Test/test_clf_qp` — picks `max` of `sort({d.name})`, *not* `max([d.datenum])`. The timestamp embedded in the filename is zero–padded, so a lexicographic sort is chronological, whereas filesystem mtime is not checkout–invariant: a fresh `git clone` or `git`

`worktree add` stamps every checked–out file with the same time, and an mtime `max` then breaks the tie arbitrarily — in practice returning the *oldest* gait. Selecting by mtime therefore silently seeds a solve from a stale result and makes the outcome depend on which checkout it runs in.

- **Figures.** The result figures in this document (Figures 1, 2, 4, 3) are not hand–drawn: they are produced by `Visualization/make_doc_figures.m` from the newest `Results/col_result_*.mat` — here `col_result_2026-07-23_00-45-04.mat` — into `docs/figures/`. After the reference gait changes, re–run that script and recompile (`pdflatex` twice, for the cross–references) so the figures and the prose cannot drift apart.
- **Nondeterminism.** `make_random_initial_state.m` uses `randn` with no seed. The HZD pipeline instead uses a hard–coded x_0, so its results are reproducible. Add `rng(seed)` before random sampling if determinism is needed there.
- **Orphaned cache.** `Trajectory_Optimization/Trajectory.mat` is committed but loaded by no code path; treat as unverified.
- **Simscape.** `Dynamics/Test_dynamics_Simscape.slx` is a standalone dynamics cross–check, not part of the runtime pipeline.

13 Known issues and broken legacy entry points

The following predate the HZD pipeline and **do not run as–is** — they call functions that no longer exist. They are retained for reference only.

- `main_demo.m`: calls `make_initial_state` expecting 3 outputs (it returns 1), `simulate_n_steps` with 4 arguments (it takes 3), and `animate_rabbit_stepping_stones` (does not exist).
- `Controller/rabbit_controller.m`: calls `desired_gait` / `desired_gait_velocity` (do not exist).
- `Dynamics/rabbit_dynamics.m`: calls `D_matrix` / `C_vector` / `G_vector` (do not exist; the real terms are `M` / `V` / `G`).

Open item: stance–foot constraint drift (a symptom). The stance foot is held only by the acceleration–level contact constraint, so `ode45` permits numerical drift of the foot within a step ($\sim$ 0.1–0.3 m; `inspect_solution.m` reports it). The natural fix is Baumgarte stabilization (`rabbit_constrained_dynamics.m` supports it via an optional `foot_ref`), but on the reference gait it *diverged* rather than helping. That is not a flaw in Baumgarte: as Section 8 shows, the seed orbit has $\rho \approx 3$, so it amplifies the very corrections Baumgarte injects. The drift is thus best read as *another symptom of orbital instability*; enforcing $\rho < 1$ addresses the root cause, after which constraint stabilization becomes well behaved.

Open item: enforcing stability is expensive. The $\rho < 1$ constraint is finite–difference heavy (Section 8). It is usable as a warm–started final phase but slow for a from–scratch solve; a reduced (2–dimensional hybrid zero dynamics) stability condition, or an analytic Poincaré Jacobian, would make it cheap. This is the natural next step for the pipeline, alongside the CLF–QP controller route (which stabilizes an open–loop–unstable orbit with feedback instead of redesigning it).

References

1. E. R. Westervelt, J. W. Grizzle, C. Chevallereau, J. H. Choi, B. Morris, *Feedback Control of Dynamic Bipedal Robot Locomotion*, CRC Press, 2007.